Thread Counter - Result Analysis

Java program: Ashik_Thread | Usage: java Ashik_Thread <threads> <increments> <thread-safe>

1. Detailed Test Results

Test Case	Threads	Incr. / Thread	Expected Count	Mode	Static Count	Non-Static Total	Abs. Difference	% Difference	Exec. Time (ms)
TC_01	1	1,000	1,000	Safe (true)	1,000	1,000	0	0.0000%	2.585
				Unsafe (false)	1,000	1,000	0	0.0000%	1.866
TC_02	2	10,000	20,000	Safe (true)	20,000	20,000	0	0.0000%	4.455
				Unsafe (false)	13,673	20,000	6,327	31.6350%	4.258
TC_05	5	10,000	50,000	Safe (true)	50,000	50,000	0	0.0000%	10.230
				Unsafe (false)	16,260	50,000	33,740	67.4800%	5.045
TC_10	10	10,000	100,000	Safe (true)	100,000	100,000	0	0.0000%	17.871
				Unsafe (false)	20,791	100,000	79,209	79.2090%	7.736
TC_20	20	10,000	200,000	Safe (true)	200,000	200,000	0	0.0000%	32.339
				Unsafe (false)	43,264	200,000	156,736	78.3680%	19.631
TC_50	50	10,000	500,000	Safe (true)	500,000	500,000	0	0.0000%	75.358
				Unsafe (false)	95,671	500,000	404,329	80.8658%	48.112
TC_100	100	10,000	1,000,000	Safe (true)	1,000,000	1,000,000	0	0.0000%	76.241
				Unsafe (false)	263,113	1,000,000	736,887	73.6887%	67.134

2. Summary Comparison (Safe vs Unsafe)

Threads	Lost Updates (Unsafe)	% Data Loss	Safe Time (ms)	Unsafe Time (ms)	Safety Overhead
1	0	0.00%	2.585	1.866	+0.719 ms (~38.5%)
2	6,327	31.64%	4.455	4.258	+0.197 ms (~4.6%)
5	33,740	67.48%	10.230	5.045	+5.185 ms (~102.8%)

Threads	Lost Updates (Unsafe)	% Data Loss	Safe Time (ms)	Unsafe Time (ms)	Safety Overhead
10	79,209	79.21%	17.871	7.736	+10.135 ms (~131.0%)
20	156,736	78.37%	32.339	19.631	+12.708 ms (~64.7%)
50	404,329	80.87%	75.358	48.112	+27.246 ms (~56.6%)
100	736,887	73.69%	76.241	67.134	+9.107 ms (~13.6%)

3. Key Observations

• **Thread-safe mode is always correct.** In all 7 test cases the static count equals the expected count (0 difference).

• **Non-thread-safe mode shows a race condition.** With 1 thread there is no error; from 2 threads onward updates are lost because *count*++ is not atomic.

• **Data loss grows quickly.** Error rises from 31.6% (2 threads) to about 67% (5 threads), then stays around 74-81%.

• **Non-static total is always correct.** Each thread uses its own local counter, so there is no shared state.

• **Synchronization has a cost.** Safe mode is slower in every case; overhead peaks at 5-10 threads (~100-130%).

• **Execution time scales with workload** as total increments grow from 1,000 to 1,000,000.

4. Conclusion

Unsynchronized access to a shared static counter causes lost updates (race conditions) once more than one thread is used, while synchronization guarantees correctness at the cost of some execution time. Correctness should take priority over the small performance gain of the unsafe version.